An adaptive wavelet-based PINN for problems with localized high-magnitude source

Himanshu Pandey[1] and **Ratikanta Behera**[1]

[1] *Department of Computational and Data Sciences, Indian Institute of Science, Bangalore, 560012, India*

Abstract. In recent years, physics-informed neural networks (PINNs) have gained significant attention for solving differential equations, although they suffer from two fundamental limitations, namely, spectral bias inherent in neural networks and loss imbalance arising from multiscale phenomena. This paper proposes an adaptive wavelet-based PINN (AW-PINN) to address the extreme loss imbalance characteristic of problems with localized high-magnitude source terms. Such problems frequently arise in various physical applications, such as thermal processing, electro-magnetics, impact mechanics, and fluid dynamics involving localized forcing. The proposed framework dynamically adjusts the wavelet basis function based on residual and supervised loss. This adaptive nature makes AW-PINN handle problems with high-scale features effectively without being memory-intensive. Additionally, AW-PINN does not rely on automatic differentiation to obtain derivatives involved in the loss function, which accelerates the training process. The method operates in two stages, an initial short pre-training phase with fixed bases to select physically relevant wavelet families, followed by an adaptive refinement that adapts scales and translations without populating high-resolution bases across entire domains. Theoretically, we show that under certain assumptions, AW-PINN admits a Gaussian process limit and derive its associated NTK structure. We evaluate AW-PINN on several challenging PDEs featuring localized high-magnitude source terms with extreme loss imbalances having ratios up to $10^{10} : 1$. Across these PDEs, including transient heat conduction, highly localized Poisson problems, oscillatory flow equations, and Maxwell's equations with a point charge source, AW-PINN consistently outperforms existing methods in its class.

Keywords: Deep learning, Physics-informed machine learning, Loss balancing, Wavelet bases

1 Introduction

Recent years have witnessed significant progress in learning-based methods, mainly due to enhanced computational capabilities, advanced optimization algorithms, and the development of automatic differentiation techniques [1, 2]. These advances have facilitated the widespread adoption of deep learning approaches within the scientific community. Among these, physics-informed neural networks (PINNs) [3] have emerged as a promising tool for solving partial differential equations (PDEs) in both forward and inverse settings. PINNs provide a meshless framework that optimizes the loss function by incorporating physical information through the residuals of differential equations. This methodology offers a robust alternative to conventional numerical methods, especially for tackling high-dimensional PDEs [4] and problems defined on complex geometries [5]. In particular, the capability of PINNs stands out when addressing inverse problems and generating highly resolved solutions by generalizing solutions across a complete computational domain rather than discrete points. Additionally, the PINN framework has been successfully extended to address integro-differential equations [6], stochastic differential equations [7, 8], and fractional differential equations [9, 10]. For a comprehensive overview of the theory and applications of PINNs, refer to [11, 12, 13] and the references therein.

Although PINNs offer notable advantages, several limitations have been extensively discussed and addressed in the literature. One key limitation arises from the spectral bias inherent in neural networks [14, 15], which causes PINNs to preferentially learn low-frequency components of the solution. Jacot *et al.* [16] formulated the neural tangent kernel (NTK) theory, demonstrating that the training process of an infinite-width neural network is equivalent to kernel regression with the NTK. Their analysis shows that loss function components associated with larger eigenvalues converge more rapidly, and the eigenvalues decrease sharply as the frequency of the target function increases. Building on NTK theory, Wang *et al.* [17] investigated the training dynamics of PINNs and confirmed the presence of spectral bias in PINNs. Additionally, the work by Wang *et al.* [18] revealed that the gradient flow in PINN models becomes stiff for PDEs with high-frequency components, resulting in imbalanced gradients during backpropagation. To overcome the challenge of learning high-frequency components, Tancik *et al.* [19] proposed Fourier feature mapping, which enhances the network's ability to represent high-frequency functions. Building on this approach, Wang *et al.* [20] demonstrated that Fourier feature mapping modulates the frequency of NTK eigenvectors and developed an improved PINN architecture by integrating Fourier features before the fully connected layers. This architecture enhances the capacity of PINNs to solve PDEs with pronounced high-frequency characteristics. Further advancements include the work of Liu *et al.* [21], which used multiple Fourier feature encoding and adaptively adjusted encoding frequencies based on the loss function.

In addition to spectral bias, PINNs also encounter poor training dynamics when there is a significant magnitude disparity between different loss terms. This loss imbalance is prominent in multiscale PDEs, where the optimizer tends to minimize the most dominant loss term at the expense of others. Such an imbalance hinders the ability of the model to accurately learn and represent all relevant features of the solution. Several studies have been conducted to address the loss balancing issue in PINNs. McClenny *et al.* proposed Self-adaptive PINNs (SA-PINNs) [22] that use trainable self-adaptive weights in loss terms at each training point. During optimization, the loss is minimized with respect to the network weights and maximized with respect to the self-adaptive weights. A non-negative, strictly increasing, and differentiable mask function, which takes self-adaptive weights as an argument, is used to make the network pay more attention to training points that are hard to fit. To further address the issue of loss term imbalance, Wang *et al.* [23] developed a modified loss function by imposing different power operations on each loss term to make them of the same order of magnitude. Further, Bischof *et al.* [24] developed relative loss balancing with random look-back, in which loss weights are updated based on loss statistics from previous training steps using exponential decay. Additionally, a Bernoulli random variable is used to decide whether relative improvements since the beginning of the training should be carried forward. This random look-back helps the model escape local minima by changing the loss space. These loss balancing techniques require careful selection of hyperparameters or prior knowledge of the solution to appropriately scale the loss terms for optimal performance. Alternatively, rather than explicitly balancing the loss function, Pandey *et al.* introduced wavelet-based PINNs (W-PINNs) [25], which train the model in wavelet space and bring it back to physical space using precomputed wavelet matrices. In wavelet space, the multiscale nature of the problem is smoothed out, making the optimization effective. Eventually, it transforms the optimization process to learn weights corresponding to wavelet bases composed of various scales and translates, inherently addressing both loss balancing and the spectral bias nature of the PINN. Furthermore, W-PINNs do not rely on automatic differentiation for derivatives involved in the loss function, which significantly accelerates the training process. However, this spectral approach is promising, but managing wavelet matrices, particularly for problems involving extremely high-scale features, becomes memory-intensive. Moreover, the exponential growth in the number of wavelet basis functions with increasing

scales further increases optimization challenges.

In the present study, we propose an adaptive wavelet-based PINN (AW-PINN), which extends the W-PINN framework by enabling the network to dynamically adapt the wavelet family, rather than relying on a fixed basis. This adaptive nature enhances the effectiveness of the method in resolving problems with high-scale features. In particular, we focus on problems with localized high-magnitude source terms. Such problems frequently arise in various physical contexts, including heat conduction in materials with spatially localized sources, like in welding, electromagnetic wave propagation in inhomogeneous media, and fluid dynamics involving localized forcing. The rest of the manuscript is organized into four sections. Starting with Section 2, which reviews several related methodologies, we later compare the performance of our method against these. Followed by Section 3 describes the proposed AW-PINN framework in detail, and subsequently, the testing over a couple of challenging problems in Section 4. Finally, Section 5 summarizes the main findings along with the method's limitations, and potential directions for future research.

2 Related Works

2.1 *PINN framework for problems with multi-magnitude loss terms*

Wang *et al.* [23] developed a PINN framework for multiscale problems with multi-magnitude loss terms (MMPINN). To balance the magnitude of different loss terms, they proposed a regularization strategy that applies power operations on each loss term. To further refine their strategy, they incorporate multi-level training by adjusting the power exponent at each level and using a grouping regularization for different subdomains. The effectiveness of this regularization scheme is demonstrated through several PDE examples exhibiting loss imbalance, including a few with strong localized source terms. This makes it a fair comparison benchmark for our proposed method.

Consider the following general form of PDE

$$\begin{cases} \mathscr{P}[u(\boldsymbol{x})] = f(\boldsymbol{x}), & \boldsymbol{x} \in \Omega, \\ \mathscr{B}[u(\boldsymbol{x})] = g(\boldsymbol{x}), & \boldsymbol{x} \in \partial\Omega, \end{cases} \tag{1}$$

where $\mathscr{P}[\cdot]$ represents a differential operator on the d-dimensional domain $\Omega \subset \mathbb{R}^d$, which also includes the time variable as a component, f is the source term, and $\mathscr{B}$ corresponds to the given initial and boundary conditions. Let $\hat{u}(\boldsymbol{x};\boldsymbol{\theta})$ represent the PINN-based approximation to the PDE 1, with $\boldsymbol{\theta}$ denoting the trainable parameters of the model. These parameters, which include weights and biases of the network, are obtained by optimizing the loss function. For the traditional PINN, the loss function can be expressed as follows

$$\begin{cases} \mathcal{L}(\boldsymbol{\theta}) = \omega_r \, \mathcal{L}_{res}(\boldsymbol{\theta}) + \omega_s \, \mathcal{L}_{sup}(\boldsymbol{\theta}), \\ \mathcal{L}_{res}(\boldsymbol{\theta}) = \dfrac{1}{N_{res}} \displaystyle\sum_{i=1}^{N_{res}} \left| \mathscr{P}[\hat{u}(\boldsymbol{x}_i^r;\boldsymbol{\theta})] - f(\boldsymbol{x}_i^r) \right|^2, \quad x_i^r \in \Omega, \\ \mathcal{L}_{sup}(\boldsymbol{\theta}) = \dfrac{1}{N_{sup}} \displaystyle\sum_{i=1}^{N_{sup}} \left| \mathscr{B}[\hat{u}(\boldsymbol{x}_i^s;\boldsymbol{\theta})] - g(\boldsymbol{x}_i^s) \right|^2, \quad x_i^s \in \partial\Omega, \end{cases} \tag{2}$$

where the total loss $\mathcal{L}$ is composed of the residual loss, $\mathcal{L}_{res}$, and the supervised loss $\mathcal{L}_{sup}$. N_{res} and N_{sup} are the numbers of residual and supervised training points in Ω and $\partial\Omega$, respectively. This loss function is generally minimized using the most popular optimization algorithms, namely, Adam [26] and L-BFGS [27].

For highly imbalanced loss terms, MMPINN proposes an exponent regularized loss function, for which the loss function takes the following form

$$\bar{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}_{res}^{\frac{1}{p}}(\boldsymbol{\theta}) + \mathcal{L}_{sup}^{\frac{1}{q}}(\boldsymbol{\theta}), \quad p, q \in \mathbb{Z}^+, \tag{3}$$

where p and q are positive integer regularization parameters, which are chosen in a way that balances the order of magnitude of different loss terms. However, the effectiveness of this regularization depends on choosing p and q within an optimal range that is problem-specific.

2.2 *Wavelet-based PINN*

The Wavelet-based PINN (W-PINN) [25] learns the solution in wavelet space spanned by wavelet basis functions at multiple scales and translations that cover the entire domain. The learned solution is then mapped back to physical space using predefined wavelet basis transformation matrices. Let $\Omega = \prod_{n=1}^{d}[a_n, b_n] \subset \mathbb{R}^d$ be a compact domain. For a mother wavelet ψ on $\mathbb{R}$, the scaled and translated wavelets can be defined as

$$\psi_{j,k}(x) \;=\; 2^{j/2}\,\psi\big(2^j x - k\big)\,, \qquad j, k \in \mathbb{Z}, \tag{4}$$

where j and k are scale and translation parameters. This can be extended in d-dimensions by using a separable tensor-product basis in the following manner

$$\Psi_{\boldsymbol{j},\boldsymbol{k}}(\boldsymbol{x}) = \prod_{n=1}^{d} 2^{j_n/2}\,\psi\left(2^{j_n}x_n - k_n\right), \qquad \boldsymbol{x} \in \Omega, \;\; \boldsymbol{j} = (j_1, \ldots, j_d), \; \boldsymbol{k} = (k_1, \ldots, k_d) \in \mathbb{Z}^d. \tag{5}$$

The scale parameters are chosen from a finite resolution-level sets $J_n = \{J_{n,1}, \ldots, J_{n,N_n}\} \subset \mathbb{Z}$ for each coordinate $n = 1, \ldots, d$. For each $j_n \in J_n$, define the integer translation range

$$K_n(j_n) \;:=\; \Big\{ k \in \mathbb{Z} \;:\; \big\lfloor (a_n - \gamma)\, 2^{j_n} \big\rfloor \;\le\; k \;\le\; \big\lceil (b_n + \gamma)\, 2^{j_n} \big\rceil \Big\}, \tag{6}$$

which ensures coverage of Ω at each resolution. Here $\gamma > 0$ is a fixed translation hyperparameter. For this hierarchical multi-resolution basis, we can define a multi-index set as

$$\mathcal{I} \;=\; \Big\{ (\boldsymbol{j}, \boldsymbol{k}) \in \mathbb{Z}^d \times \mathbb{Z}^d \;:\; j_n \in J_n,\; k_n \in K_n(j_n) \text{ for } n = 1, \ldots, d \Big\}. \tag{7}$$

Consider a bijection mapping $\mathcal{M} : \mathcal{I} \to \{1, \ldots, N_{\text{fam}}\}$, and define $\Psi_i(\boldsymbol{x}) := \Psi_{\boldsymbol{j},\boldsymbol{k}}(\boldsymbol{x})$ for $i = \mathcal{M}(\boldsymbol{j}, \boldsymbol{k})$. Let $\hat{u}^{\psi}(\boldsymbol{x}; \boldsymbol{\theta})$ is a W-PINN approximation to the PDE 1 such that

$$\hat{u}^{\psi}(\boldsymbol{x}; \boldsymbol{\theta}) = \sum_{i=1}^{N_{fam}} c_i(\boldsymbol{\theta})\Psi_i(\boldsymbol{x}) + \mathcal{B}, \quad \boldsymbol{x} \in \Omega \cup \partial\Omega, \tag{8}$$

where $\boldsymbol{\theta} \cup \{\mathcal{B}\}$ are the trainable parameters of W-PINN network for learnable coefficients $\{c_i\}_{i=1}^{N_{\text{fam}}}$, and the fixed basis $\{\Psi_i\}_{i=1}^{N_{\text{fam}}}$. Training minimizes the loss in Equation (2), with $\hat{u}$ replaced by $\hat{u}^{\psi}$ from (8). Instead of using autograd, derivatives required in the loss function are obtained by analytically differentiating the basis functions Ψ_i, i.e,

$$\partial_{x_m} \Psi_{\boldsymbol{j},\boldsymbol{k}}(\boldsymbol{x}) = 2^{j_m}\, 2^{j_m/2}\, \psi'\left(2^{j_m}x_m - k_m\right) \times \prod_{n \neq m} 2^{j_n/2}\, \psi\left(2^{j_n}x_n - k_n\right), \tag{9}$$

and higher-order derivatives follow similarly. The W-PINN effectively addresses multiscale PDEs because the localized wavelet basis transforms the problem into wavelet space, where multiscale features become smoother, eventually facilitating the training process.

2.3 *Neural Tangent Kernel theory for PINN*

The neural tangent kernel (NTK) [16, 17] provides a theoretical perspective for training dynamics of neural networks. Consider an infinite width PINN approximating the PDE (1) with $\hat{u}(\boldsymbol{x};\boldsymbol{\theta})$. For gradient descent algorithm with a sufficiently small learning rate, Wang *et al.* [17] showed that, in the infinite-width limit, the training dynamics converge to independent, identically distributed Gaussian processes with zero mean. Based on this, NTK for PINN is defined as

$$\boldsymbol{\mathcal{K}}(t) = \begin{bmatrix} \boldsymbol{\mathcal{K}}_{pp}(t) & \boldsymbol{\mathcal{K}}_{pb}(t) \\ \boldsymbol{\mathcal{K}}_{bp}(t) & \boldsymbol{\mathcal{K}}_{bb}(t) \end{bmatrix}, \tag{10}$$

where t is the training step. For given training points $\{\boldsymbol{x}_i^r, f(\boldsymbol{x}_i^r)\}$ and $\{\boldsymbol{x}_i^s, g(\boldsymbol{x}_i^s)\}$, $\{i,j\}$-th element of submatrices of the NTK matrix $\boldsymbol{\mathcal{K}}$ is obtained by

$$\begin{aligned}
\left(\boldsymbol{\mathcal{K}}_{pp}\right)_{ij}(t) &= \left\langle \frac{\partial \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}_i^r;\boldsymbol{\theta}(t)\right)\right]}{\partial \boldsymbol{\theta}}, \frac{\partial \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}_j^r;\boldsymbol{\theta}(t)\right)\right]}{\partial \boldsymbol{\theta}} \right\rangle, \\
\left(\boldsymbol{\mathcal{K}}_{pb}\right)_{ij}(t) &= \left\langle \frac{\partial \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}_i^r;\boldsymbol{\theta}(t)\right)\right]}{\partial \boldsymbol{\theta}}, \frac{\partial \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}_j^s;\boldsymbol{\theta}(t)\right)\right]}{\partial \boldsymbol{\theta}} \right\rangle, \\
\left(\boldsymbol{\mathcal{K}}_{bb}\right)_{ij}(t) &= \left\langle \frac{\partial \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}_i^s;\boldsymbol{\theta}(t)\right)\right]}{\partial \boldsymbol{\theta}}, \frac{\partial \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}_j^s;\boldsymbol{\theta}(t)\right)\right]}{\partial \boldsymbol{\theta}} \right\rangle,
\end{aligned}$$

where $\langle\cdot,\cdot\rangle$ denotes the inner product over all trainable parameters. Further, using the NTK matrix, training a PINN through gradient descent can be expressed as

$$\begin{bmatrix} \dfrac{\partial \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}^r;\,\boldsymbol{\theta}(t)\right)\right]}{\partial t} \\ \dfrac{\partial \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}^s;\,\boldsymbol{\theta}(t)\right)\right]}{\partial t} \end{bmatrix} = -\boldsymbol{\mathcal{K}}(t) \begin{bmatrix} \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}^r;\,\boldsymbol{\theta}(t)\right)\right] - f\left(\boldsymbol{x}^r\right) \\ \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}^s;\,\boldsymbol{\theta}(t)\right)\right] - g\left(\boldsymbol{x}^s\right) \end{bmatrix}. \tag{11}$$

Under the assumption of training a sufficiently large-width PINN with a small enough learning rate, Wang *et al.* [17] showed the NTK remains constant, ie, $\boldsymbol{\mathcal{K}}(t) \approx \boldsymbol{\mathcal{K}}(0)$. Using this solution of the Equation (11) becomes

$$\begin{bmatrix} \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}^r;\,\boldsymbol{\theta}(t)\right)\right] - f\left(\boldsymbol{x}^r\right) \\ \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}^s;\,\boldsymbol{\theta}(t)\right)\right] - g\left(\boldsymbol{x}^s\right) \end{bmatrix} \approx -e^{-\boldsymbol{\mathcal{K}}(0)t} \begin{bmatrix} f\left(\boldsymbol{x}^r\right) \\ g\left(\boldsymbol{x}^s\right) \end{bmatrix}, \tag{12}$$

which provides error evolution during training, which can be further simplified by spectral decomposition of the NTK: $\boldsymbol{\mathcal{K}}(0) = \boldsymbol{\mathcal{Q}}^T \boldsymbol{\Lambda} \boldsymbol{\mathcal{Q}}$, here $\boldsymbol{\mathcal{Q}}$ and $\boldsymbol{\Lambda}$ are orthogonal matrix and corresponding diagonal eigenvalue matrix. Then the Equation (13) take the form

$$\begin{bmatrix} \mathscr{P}\left[\hat{u}\left(\boldsymbol{x}^r;\,\boldsymbol{\theta}(t)\right)\right] - f\left(\boldsymbol{x}^r\right) \\ \mathscr{B}\left[\hat{u}\left(\boldsymbol{x}^s;\,\boldsymbol{\theta}(t)\right)\right] - g\left(\boldsymbol{x}^s\right) \end{bmatrix} \approx -\boldsymbol{\mathcal{Q}}^T e^{-\boldsymbol{\Lambda} t} \boldsymbol{\mathcal{Q}} \begin{bmatrix} f\left(\boldsymbol{x}^r\right) \\ g\left(\boldsymbol{x}^s\right) \end{bmatrix}. \tag{13}$$

This formulation suggests that the i^{th} error component exhibits exponential decay at a rate determined by the corresponding NTK eigenvalue; hence, larger eigenvalues yield faster decay of the error and, consequently, faster convergence of the PINN.

3 Adaptive Wavelet-based PINN

The Adaptive Wavelet-based PINN (AW-PINN) extends the W-PINN framework by making the wavelet family adaptive, allowing scales and translates to be learned as parameters, unlike the fixed values in W-PINN. The method consists of two stages: a short pre-training stage using the W-PINN formulation 2.2, followed by an adaptive stage that refines the selected wavelet families obtained from the pre-training stage. We discuss both stages in detail below.

- *Pre-training and family selection*

  Consider W-PINN pre-training for the PDE (1) with residual points $\{x_i^r\}_{i=1}^{N_{res}}$ and supervised points $\{x_i^s\}_{i=1}^{N_{sup}}$. Evaluate the right side of PDE (1) at the residual and supervised training points in vector form

$$\begin{aligned} \mathbf{f} &= (f(\boldsymbol{x}_1^r), \ldots, f(\boldsymbol{x}_{N_{res}}^r))^\top \in \mathbb{R}^{N_{res}}, \\ \mathbf{g} &= (g(\boldsymbol{x}_1^s), \ldots, g(\boldsymbol{x}_{N_{res}}^s))^\top \in \mathbb{R}^{N_{sup}}, \end{aligned} \tag{14}$$

  where f and g denote the PDE residual source term and supervised condition values, respectively. For each wavelet family index i with basis Ψ_i and pre-trained coefficient c_i, define the family-induced residual and boundary response vectors

$$\begin{aligned} \mathbf{R}_i &= \left(\mathscr{P}[c_i\Psi_i(\boldsymbol{x}_1^r), \ldots, \mathscr{P}[c_i\Psi_i(\boldsymbol{x}_{N_{res}}^r)\right)^\top \in \mathbb{R}^{N_{res}}, \\ \mathbf{B}_i &= \left(\mathscr{B}[c_i\Psi_i(\boldsymbol{x}_1^s), \ldots, \mathscr{B}[c_i\Psi_i(\boldsymbol{x}_{N_{sup}}^s)\right)^\top \in \mathbb{R}^{N_{sup}}, \end{aligned} \tag{15}$$

  then we get the alignment measure between the family i and the PDE right side via the following similarity scores

$$\text{Score}_i^R = \frac{\langle \mathbf{R}_i,\ \mathbf{f}\rangle}{|\langle \mathbf{R}_i,\ \mathbf{f}\rangle|_{max}}, \qquad \text{Score}_i^B = \frac{\langle \mathbf{B}_i,\ \mathbf{g}\rangle}{|\langle \mathbf{B}_i,\ \mathbf{g}\rangle|_{max}}, \tag{16}$$

  When the right-hand side of the PDE is identically zero, the above similarity becomes ill-defined. In such cases, mean of the normalized response vector, $\bar{\mathbf{R}}_i/\|\mathbf{R}_i\|_{max}$ and $\bar{\mathbf{B}}_i/\|\mathbf{B}_i\|_{max}$, are used as respective scores. Wavelet families exhibiting a higher similarity score or lower normalized response magnitudes are deemed more physically relevant. Additionally, wavelet families corresponding to the top κ largest coefficient magnitudes $|c_i|$ are also included. This similarity-based selection works because the wavelet bases exhibit minimal overlap. The final set of active families is denoted as

$$\mathcal{I}_{\text{A}} = \left\{ (\boldsymbol{j}, \boldsymbol{k}) \in \mathbb{Z}^d \times \mathbb{Z}^d \right\} \subset \mathcal{I}, \tag{17}$$

  along with their corresponding coefficients $\boldsymbol{c}_{\text{A}} = \{c_i\}_{i=1}^{N_{\text{A}}}$, N_{A} denotes number of selected wavelet basis function.

- *Adaptive stage and network formulation*

  The selected coefficients and scale-translate parameters are utilized to initialize the AW-PINN network. For each selected family $i \in \mathcal{I}_{\text{A}}$ associated with scale–translate pair $(\boldsymbol{j}_i, \boldsymbol{k}_i)$, an adaptive submodule $\mathcal{W}_i(\boldsymbol{x}; \boldsymbol{\theta}_i)$ is constructed as a parametric wavelet. For d-dimensional input $\boldsymbol{x} = (x_1, \ldots, x_d)$, we define

$$\mathcal{W}_i(\boldsymbol{x}; \boldsymbol{\theta}_i) = \prod_{n=1}^{d} \psi(w_{i,n}\, x_n + b_{i,n})\,, \qquad \boldsymbol{\theta}_i = \{(w_{i,n}, b_{i,n})\}_{n=1}^{d}, \tag{18}$$

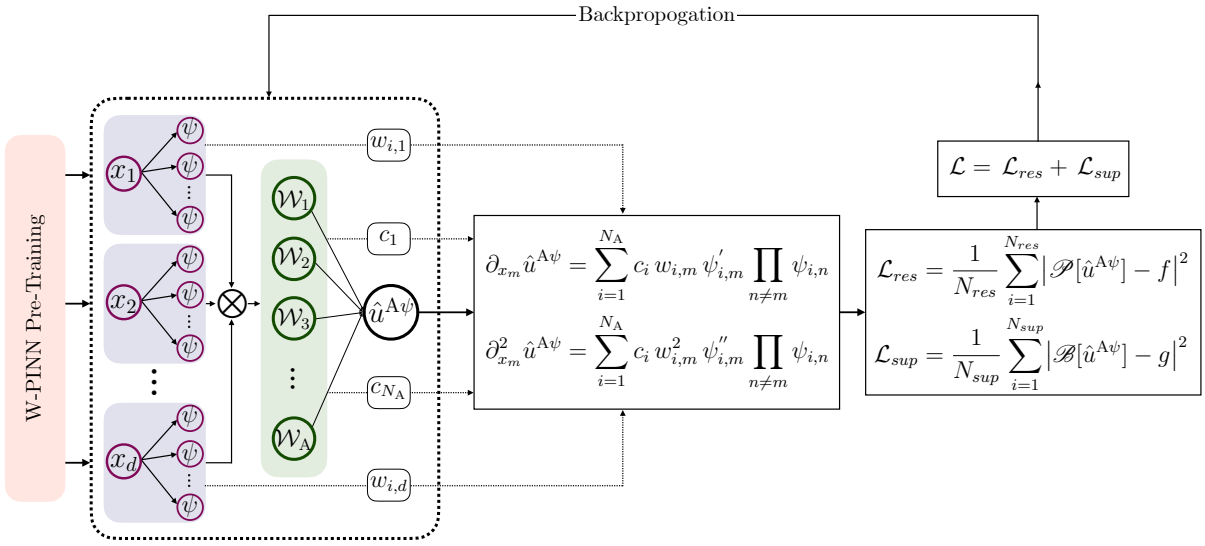

Fig. 3.1. A schematic architecture of AW-PINN. Parameters for purple wavelet units and final linear layer are initialized using $\mathcal{I}_{\text{adapt}}$ and $\boldsymbol{c}_{\text{adapt}}$, respectively. The symbol ψ denotes the wavelet activation function and $\psi_{i,m} = \psi(w_{i,m}x_m + b_{i,m})$, while $\bigotimes$ represents element-wise multiplication.

where $\psi(\cdot)$ is the same mother wavelet used in the Equation (4). The initial parameters are transferred from $\mathcal{I}_{\text{A}}$ as, $w_{i,n}^{(0)} = 2^{j_{i,n}}, b_{i,n}^{(0)} = -k_{i,n}$, enabling continuous adaptation of scale and translation during optimization. This adaptive wavelet layer is achieved by using $\psi(\cdot)$ as activation function. A final linear layer, initialized with $\boldsymbol{c}_{\text{A}}$, is imposed to get AW-PINN approximation

$$\hat{u}^{A\psi}(\boldsymbol{x};\boldsymbol{\theta}) = \sum_{i=1}^{N_A} c_i(\boldsymbol{\theta})\, \mathcal{W}_i(\boldsymbol{x};\boldsymbol{\theta}_i) + \mathcal{B}, \tag{19}$$

where $\mathcal{B}$ denotes a trainable bias term. The optimization process minimizes the same

Algorithm 1 Adaptive Wavelet-based PINN (AW-PINN)

Input: $\mathscr{P}, \mathscr{B}$, domain Ω, points $\{x_i^r\}, \{x_i^s\}$, wavelet resolutions $\{J_n\}$, translation hyperparameter γ, threshold κ.

Stage 1: Pre-training and family selection

1: Build fixed family $\mathcal{I} = \{(\mathbf{j}, \mathbf{k}) : j_n \in J_n,\ k_n \in K_n(j_n)\}$, as described in [25].
2: Pre-train W-PINN $\hat{u}^{\psi}(\boldsymbol{x};\boldsymbol{\theta}) = \sum c_i(\boldsymbol{\theta})\Psi_i(\boldsymbol{x}) + \mathcal{B}$ via loss (2).
3: For each $i \in \mathcal{I}$ compute responses $\mathbf{R}_i = \mathscr{P}[c_i\Psi_i]$, $\mathbf{B}_i = \mathscr{B}[c_i\Psi_i]$.
4: Compute similarity scores $\text{Score}_i^R = \langle \mathbf{R}_i, \mathbf{f}\rangle / |\langle \mathbf{R}_i, \mathbf{f}\rangle|_{\max}$, $\text{Score}_i^B = \langle \mathbf{B}_i, \mathbf{g}\rangle / |\langle \mathbf{B}_i, \mathbf{g}\rangle|_{\max}$.
5: Select active families $\mathcal{I}_A :=$ selection based on score threshold and top κ coefficients $|c_i|$.

Stage 2: Adaptive stage

6: Initialize adaptive parameters for $i \in \mathcal{I}_A$: $w_{i,n} = 2^{j_{i,n}}$, $b_{i,n} = -k_{i,n}$.
7: Define adaptive wavelet unit $\mathcal{W}_i(\boldsymbol{x};\boldsymbol{\theta}_i) = \prod_{n=1}^{d} \psi(w_{i,n}x_n + b_{i,n})$.
8: Initialize c_i from pre-training and construct $\hat{u}^{A\psi}(\boldsymbol{x};\boldsymbol{\theta}) = \sum_{i\in\mathcal{I}_A} c_i(\boldsymbol{\theta})\, \mathcal{W}_i(\boldsymbol{x};\boldsymbol{\theta}_i) + \mathcal{B}$.
9: Compute analytic derivatives of $\mathcal{W}_i$ for $\mathscr{P}$ and $\mathscr{B}$.
10: Train model parameters $\boldsymbol{\theta} = \{c_i, \boldsymbol{\theta}_i, \mathcal{B}\}$ by minimizing loss in Eq. (2) using L-BFGS until convergence.

Output: Trained solution $\hat{u}^{A\psi}(\boldsymbol{x};\boldsymbol{\theta}^*)$

physics-informed loss as in the Equation (2), and the autograd is avoided by taking the analytical derivative of the Equation (18), similar to the Equation (9). A schematic diagram of the AW-PINN architecture is shown in Fig. 3.1, and the AW-PINN algorithm is summarized in Algorithm 1.

We now demonstrate that, as the adaptive family grows to an infinitely large size, under certain assumptions, the AW-PINN training converges to a zero-mean Gaussian process $\mathcal{G}_p$.

Theorem 1. *For bias $\mathcal{B} = 0$, as $N_{\mathrm{A}} \to \infty$, the random functions $\hat{u}_N^{\mathrm{A}\psi}(\cdot)$ converge in finite-dimensional distributions to a centered Gaussian process*

$$\hat{u}_{\infty}^{\mathrm{A}\psi}(\cdot) \;= \frac{1}{\sqrt{N_{\mathrm{A}}}} \sum_{i=1}^{N_{\mathrm{A}}} c_i \mathcal{W}_i \sim \; \mathcal{G}_p\big(0, k(\cdot,\cdot)\,\big)$$

Proof. Fix $m \in \mathbb{N}$ and inputs $X = \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\} \subset \mathbb{R}^d$. Define

$$\mathbf{f}_{N_{\mathrm{A}}} \;=\; \big(\hat{u}_{N_{\mathrm{A}}}^{\mathrm{A}\psi}(\boldsymbol{x}^{(1)}),\; \ldots,\; \hat{u}_{N_{\mathrm{A}}}^{\mathrm{A}\psi}(\boldsymbol{x}^{(m)})\big)^{\top} = \frac{1}{\sqrt{N_{\mathrm{A}}}} \sum_{i=1}^{N_{\mathrm{A}}} c_i\, \boldsymbol{\Phi}_i,$$

where the random feature vector

$$\boldsymbol{\Phi}_i \;=\; \big(\mathcal{W}_i(\boldsymbol{x}^{(1)};\boldsymbol{\theta}_i), \ldots, \mathcal{W}_i(\boldsymbol{x}^{(m)};\boldsymbol{\theta}_i)\big)^{\top} \in \mathbb{R}^m.$$

Assume parameters and coefficients are initialized i.i.d. normal distribution, $c_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_c^2)$, $\theta_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_\theta^2)$, where σ_c and σ_θ are fixed constants, then

$$\mathrm{Cov}\,[c_i\, \boldsymbol{\Phi}_i] = \sigma_c^2\; \mathbb{E}_{\boldsymbol{\theta}}[\boldsymbol{\Phi}_i\, \boldsymbol{\Phi}_{\boldsymbol{i}}^{\top}],$$

whose (m,n)-th entry is $\sigma_c^2\, \mathbb{E}_{\boldsymbol{\theta}}\,[\mathcal{W}(\boldsymbol{x^{(m)}};\boldsymbol{\theta})\mathcal{W}(\boldsymbol{x^{(n)}};\boldsymbol{\theta})]$.

By the multivariate central limit theorem, $\mathbf{f}_{N_{\mathrm{A}}}$ converges in distribution to a multivariate Gaussian with mean zero and covariance matrix equal to that limit covariance. Because the finite dimensional distributions converge to Gaussian ones with the above covariance, the process converges to the centered $\mathcal{G}_p$ with kernel $\mathcal{K}(\boldsymbol{x},\boldsymbol{x}') = \sigma_c^2\; \mathbb{E}_{\boldsymbol{\theta}}\big[\mathcal{W}(\boldsymbol{x};\boldsymbol{\theta})\, \mathcal{W}(\boldsymbol{x}';\boldsymbol{\theta})\big]$. □

Although, for practical purposes, the AW-PINN parameters are transferred effectively from W-PINN pre-training, the above theorem applies under random initialization as stated. The above theorem suggests the learning dynamics of AW-PINN in the infinite family regime may be analyzed via kernel regression and neural tangent kernel theory as in Section 2.3. For a scalar output $\hat{u}^{\mathrm{A}\psi}$, the empirical NTK between inputs x, x' is

$$\mathcal{K}(\boldsymbol{x},\boldsymbol{x'}) \;=\; \big\langle \nabla_\theta \hat{u}^{\mathrm{A}\psi}(\boldsymbol{x};\boldsymbol{\theta}),\; \nabla_\theta \hat{u}^{\mathrm{A}\psi}(\boldsymbol{x'};\boldsymbol{\theta})\big\rangle, \tag{20}$$

where the inner product is over all trainable scalar parameters. Decompose the gradient into derivatives with respect to the linear coefficients c_i and the wavelet parameters θ_i. Using the model and the $1/\sqrt{N_A}$ scaling,

$$\frac{\partial u^{\mathrm{A}\psi}}{\partial c_i} = \frac{1}{\sqrt{N_A}}\, \mathcal{W}_i(\boldsymbol{x};\theta_i), \tag{21}$$

$$\frac{\partial u^{\mathrm{A}\psi}}{\partial \theta_i} = \frac{1}{\sqrt{N_A}}\, c_i\, \nabla_\varphi \mathcal{W}_i(\boldsymbol{x};\theta_i). \tag{22}$$

Hence, the NTK decomposes as a sum over units

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x'}) = \frac{1}{N_A}\sum_{i=1}^{N_A} \mathcal{W}_i(\boldsymbol{x};\theta_i)\mathcal{W}_i(\boldsymbol{x'};\theta_i) + \frac{1}{N_A}\sum_{i=1}^{N_A} c_i^2 \left\langle \nabla_\theta \mathcal{W}_i(\boldsymbol{x};\theta_i),\, \nabla_\theta \mathcal{W}_i(\boldsymbol{x'};\theta_i)\right\rangle, \tag{23}$$

we denote the two contributions by $\mathcal{K}_c$ (from linear weights c_i) and $\mathcal{K}_\theta$ (from adaptive parameters):

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x'}) = \mathcal{K}_c(\boldsymbol{x}, \boldsymbol{x'}) + \mathcal{K}_\theta(\boldsymbol{x}, \boldsymbol{x'}).$$

4 Results

This section evaluates the performance of AW-PINN on several challenging problems and compares with baseline PINN, W-PINN, and MMPINN, all of which are designed to address the loss imbalance issue in PINNs. For W-PINN and MMPINN, training starts with a few epochs of the ADAM optimizer, followed by the LBFGS optimizer until convergence. In the case of AW-PINN, the first phase is done via ADAM, and the subsequent adaptive phase employs the LBFGS optimizer. In this study, we use the Gaussian wavelet, which is the first derivative of the Gaussian function. The smooth regularity and localization property of the Gaussian wavelet make it one of the optimal choices. All model parameters are initialized using Xavier's technique [28], which prevents vanishing gradients and provides stable and efficient training. The hyperparameters used for each problem are tabulated in Appendix A 5. For quantitative comparison, we use the relative L_2-Error metric over uniformly distributed test samples across the domain, which is defined as

$$\text{Relative } L_2\text{-Error} = \frac{||u(\boldsymbol{x}^t) - \hat{u}(\boldsymbol{x}^t;\boldsymbol{\theta})||_2}{||u(\boldsymbol{x}^t)||_2}, \tag{24}$$

where u and $\hat{u}$ are the exact/reference solution and the model's prediction over test samples $\boldsymbol{x}^t$, respectively. Further, to average out the model's sensitivity over parameter initialization, each experiment is repeated five to ten times independently, and reported results correspond to the average outputs of these runs. Training is carried out using PyTorch 2.6.0 + CU12.4 on an NVIDIA RTX A6000 GPU. All source code to reproduce the results of this study will be made available on request.

4.1 *Heat conduction problem with extreme heat source*

Heat conduction problems with a localized, strong heat source are often encountered in various practical applications, such as material processing, geophysics, nuclear engineering, and many others. For such problems, challenges lie in the transient nature and the intense gradients generated in the process. This problem is chosen to test the effectiveness of the proposed method for such an extreme source.

Consider the following heat conduction model

$$\begin{cases} \frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + h(x,t), \quad (x,t) \in (-1,1)\times(0,1), \\ u(x,0) = (1-x^2)\exp\big(1/(1+\epsilon)\big), \quad x \in (-1,1), \\ u(-1,t) = 0,\ u(1,t) = 0, \quad t \in (0,1], \end{cases} \tag{25}$$

the exact solution is given as $u(x,t) = (1-x^2)\exp\big(1/((2t-1)^2+\epsilon)\big)$, for which the source term becomes

$$h(x,t) = 2\Big[1 + 2\tfrac{(2t-1)(x^2-1)}{((2t-1)^2+\epsilon)^2}\Big]\exp\Big(\tfrac{1}{(2t-1)^2+\epsilon}\Big).$$

Table 4.1 Comparison of various methods for solving the heat conduction problem 4.1.

ϵ	Method	N_I	N_B	N_R	Relative L_2-Error	Avg. Training Time
0.12	Baseline PINN	5000	10000	50000	$8.1 \pm 1.32 \times 10^{-1}$	-
	W-PINN	1000	2000	20000	$4.05 \pm 0.38 \times 10^{-5}$	15.35 min
	MMPINN	3000	6000	90000	$1.71 \pm 0.53 \times 10^{-5}$	91.21 min
	AW-PINN	**1000**	**2000**	**20000**	$\mathbf{3.52 \pm 0.81 \times 10^{-6}}$	**24.83 min**
0.11	Baseline PINN	5000	10000	50000	$9.13 \pm 0.82 \times 10^{-1}$	-
	W-PINN	1000	2000	20000	$3.11 \pm 0.63 \times 10^{-5}$	16.79 min
	MMPINN	4000	8000	100000	$2.44 \pm 0.61 \times 10^{-5}$	208.36 min
	AW-PINN	**1000**	**2000**	**20000**	$\mathbf{8.15 \pm 1.27 \times 10^{-6}}$	**24.14 min**
0.10	Baseline PINN	5000	10000	100000	$9.61 \pm 0.76 \times 10^{-1}$	-
	W-PINN	1000	2000	20000	$3.10 \pm 0.86 \times 10^{-5}$	16.28 min
	MMPINN	7000	14000	200000	$8.46 \pm 0.68 \times 10^{-1}$	-
	AW-PINN	**1000**	**2000**	**20000**	$\mathbf{8.86 \pm 0.87 \times 10^{-6}}$	**25.52 min**

For the lower ϵ, the source term as well as the solution exhibit transient behavior with a high gradient near $t = 0.5$. Additionally, due to smooth initial and boundary conditions, it poses a highly loss imbalance. For instance, at $\epsilon = 0.1$, the initial loss ratio is $\mathcal{L}_b : \mathcal{L}_i : \mathcal{L}_r = 1 : 10 : 10^9$, which makes it challenging for any PINNs-based approaches.

In our experiments, we employ 5000 Adam iterations (approximately two minutes of pre-training) using the W-PINN for pre-training, followed by L-BFGS optimization with AW-PINN until convergence. Table 4.1 lists comparative results for several small ϵ values. It includes the relative L_2-error, average training time, and number of collocation points used in training. Training times are omitted for methods with ineffective convergence. The table demonstrates AW-PINN achieves the lowest relative L_2-error for all ϵ values, with substantially reduced training time compared to MMPINN. Specifically for $\epsilon = 0.1$, only AW-PINN could solve the problem satisfactorily. Another interesting observation is that, unlike MMPINN, AW-PINN does not require additional collocation points with a decrease in ϵ values. Furthermore, for $\epsilon = 0.1$, extremely transient behavior of the heat source is evident from Fig. 4.2, and the consistently low point-wise absolute error across the domain demonstrates the effectiveness of the AW-PINN. Moreover, Fig. 4.3 shows the loss curve of AW-PINN and also compares the relative L_2-error with other methods for $\epsilon = 0.12$. The narrow shaded regions for each loss term suggest low variance in the AW-PINN model, while the relative L_2-error comparison clearly demonstrates the faster convergence achieved by AW-PINN. Additionally, the non-oscillatory error curve indicates

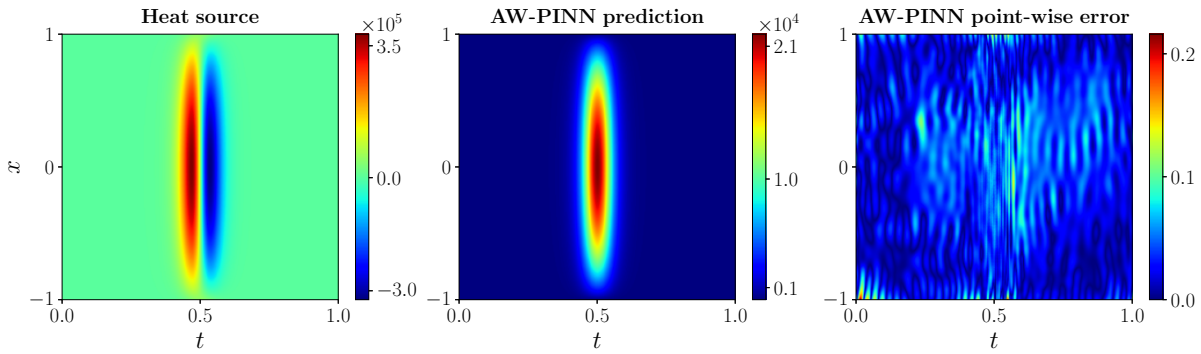

Fig. 4.2. *Left to Right:* Heat source function, the prediction using AW-PINN and respective absolute point-wise error for problem 4.1 with $\epsilon = 0.1$.

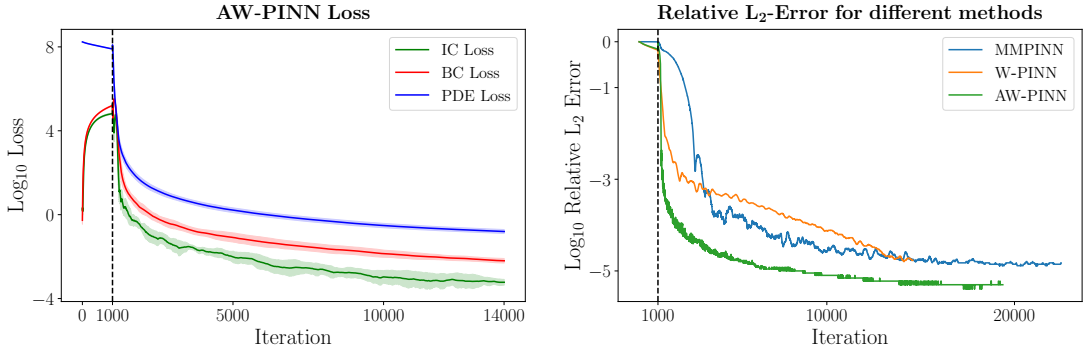

Fig. 4.3. *Left:* Average log loss plots for each loss terms in problem 4.1 with $\epsilon = 0.12$ using AW-PINN. Shaded regions denote the standard deviation across 10 independent runs. The vertical dashed line marks the completion of pre-training with W-PINN using 1000 Adam iterations.
Right: Log relative L_2-error plots of different methods for problem 4.1 with $\epsilon = 0.12$. The vertical dashed line indicates the end of the pre-training phase with 1000 Adam iterations.

that AW-PINN provides more stable training dynamics than MMPINN.

4.2 *Poisson problem with highly localized source term*

In this test case, we examine a two-dimensional Poisson equation with an extremely localized source term. Such equations frequently arise in modeling electrostatic potentials with point charges and concentration profiles in chemical diffusion processes. They are also relevant in fields such as materials science and geophysics, where localized phenomena must be accurately resolved. Consider the following Poisson model:

$$\begin{cases} \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x,y), & (x,y) \in (0,1) \times (0,1), \\ \mathscr{B}(x,y) = g(x,y), & (x,y) \in \partial\Omega, \end{cases} \tag{26}$$

with the exact solution as

$$u(x,y) = 1 + (y^2 + 10^3) \exp\left(-\frac{(x-0.5)^2}{2\epsilon^2}\right),$$

for small ϵ values, both the source function and the solution are highly localized, as illustrated in the top panel of Fig. 4.4 for $\epsilon = 0.02$. This strong localization results in a huge loss imbalance. For example, at $\epsilon = 0.02$ the initial loss ratio is $\mathcal{L}_b : \mathcal{L}_r = 10^4 : 10^{11}$, making it a challenging scenario for the PINN-based methods.

Table 4.2 compares the performance of various methods for $\epsilon = 0.05$ and $\epsilon = 0.02$. It demonstrates that the AW-PINN achieves relative L_2-error that is one and two orders of magnitude lower than those obtained by MMPINN and W-PINN, respectively. The same can be observed

Table 4.2 Comparison of various methods for solving the Poisson problem 4.2.

ϵ	Method	N_B	N_R	Relative L_2-Error	Avg. Training Time
0.05	W-PINN	4000	10000	$3.55 \pm 0.26 \times 10^{-4}$	4.87 min
	MMPINN	6000	30000	$5.71 \pm 1.53 \times 10^{-4}$	7.17 min
	AW-PINN	**4000**	**10000**	$\mathbf{3.42 \pm 0.13 \times 10^{-5}}$	**5.14 min**
0.02	W-PINN	4000	15000	$9.81 \pm 1.72 \times 10^{-3}$	4.21 min
	MMPINN	8000	50000	$3.25 \pm 1.16 \times 10^{-3}$	9.11 min
	AW-PINN	**4000**	**10000**	$\mathbf{2.68 \pm 0.53 \times 10^{-4}}$	**6.41 min**

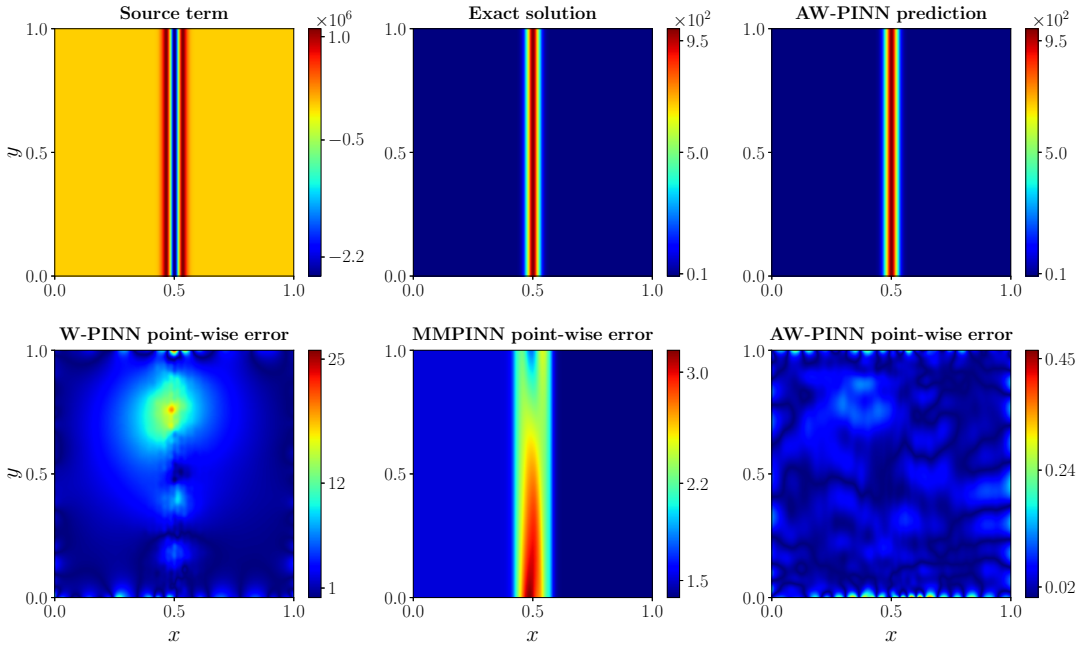

Fig. 4.4. *Top - Left to Right:* The source function, exact solution, and prediction using AW-PINN for problem 4.2 with $\epsilon = 0.02$.
Bottom - Left to Right: Absolute point-wise error of W-PINN, MMPINN, and AW-PINN, respectively.

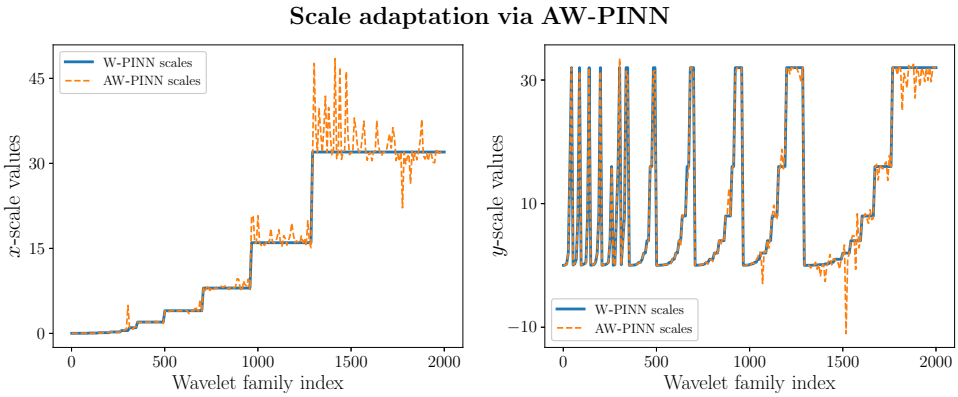

Fig. 4.5. Plot of x-scale (*l*eft) and y-scale (*r*ight) adaptation for problem 4.2. Here, solid lines denote dyadic scales and dashed lines represent scales after adaptation.

in Fig. 4.4 bottom panel, where AW-PINN exhibits the lowest absolute point-wise error across the domain. Moreover, Fig. 4.5 illustrates the scale adaptation capability of AW-PINN. In the case of W-PINN, it requires a highly resolved wavelet basis and corresponding matrices to handle such problems with localized high-scale features, which often need intensive memory and lead to highly non-convex optimization. Whereas AW-PINN efficiently adapts to higher scales only in the region where needed, without populating the entire domain with a high-resolution basis function. This not only improves computational efficiency but also enhances training stability and accuracy.

4.3 *Flow Equation with a strong oscillating source term*

The flow equation arises in a wide range of practical applications. For example, conservation laws in non-inertial reference frames, such as flows around airfoils, two-phase flow models incor-

Table 4.3 Comparison of various methods for solving the flow equation 4.3.

Method	N_I	N_B	N_R	Relative L_2-Error	Avg. Training Time
W-PINN	1000	1000	20000	$1.27 \pm 0.15 \times 10^{-2}$	4.17 min
MMPINN	2000	2000	50000	$8.35 \pm 0.97 \times 10^{-2}$	7.21 min
AW-PINN	**1000**	**1000**	**20000**	$\mathbf{5.17 \pm 0.61 \times 10^{-4}}$	**6.17 min**

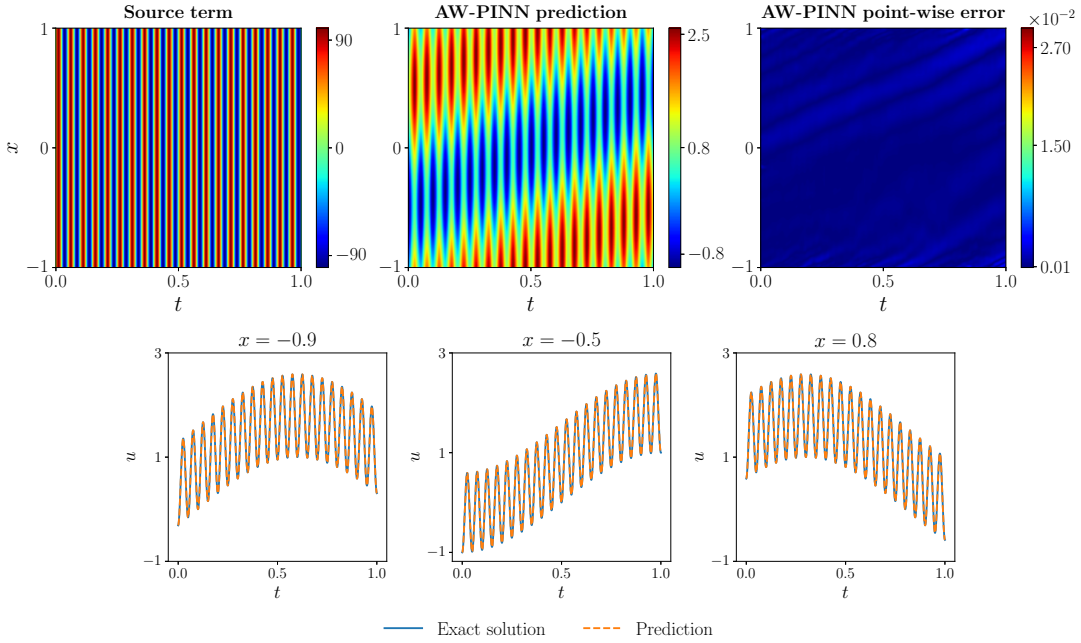

Fig. 4.6. *Top - Left to Right:* The source function, the prediction using AW-PINN, and the corresponding absolute point-wise error for problem 4.3.
Bottom: Cross-section comparison of the prediction with the exact solution at various x-domain snapshots.

porating momentum and energy exchange, turbulence models governed by two-equation formulations, and reactive flows involving chemical processes. For this example, we are interested in the following flow model with a large oscillating source term

$$\begin{cases} \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = A \sin\left(2\pi \frac{t}{T_s}\right), & (x,t) \in (-1,1) \times (0,1), \\ u(x,0) = \sin(\pi x), & x \in (-1,1), \end{cases} \tag{27}$$

with exact solution

$$u(x,t) = \sin(\pi(x-t)) + \frac{AT_s}{2\pi}\left(1 - \cos\left(2\pi \frac{t}{T_s}\right)\right),$$

here we take $A = 100$ and $T_s = 0.05$. A small T_s makes the source scale much smaller than the mean flow time scale, and a large A strengthens the oscillations of the source term. These characteristics make this flow equation a challenging problem.

The comparison Table 4.3 suggests that AW-PINN achieves accuracy superior to other methods by two orders of magnitude. The top panel of Fig. 4.6 shows a highly oscillatory source term, AW-PINN prediction, and the corresponding absolute point-wise error, demonstrating the model's efficiency. The bottom panel further validates the accuracy of the prediction through multiple spatial snapshot plots.

4.4 *Maxwell's equation for a point charge source*

In this example, we apply AW-PINN on a three-dimensional (one temporal and two spatial) electromagnetic problem. The fundamental principles of electromagnetism are governed by Maxwell's equations. It models the electric and magnetic fields through a set of coupled partial differential equations. These equations are widely used in various fields of physics and engineering, including wireless communications and antenna design, microwave devices, radar and remote sensing, photonics and optical waveguides, and biomedical imaging.

To validate the robustness of the proposed method, we consider the TEz formulation of Maxwell's equations in a rectangular cavity with perfectly electrically conducting (PEC) boundary conditions

$$\begin{aligned} \frac{\partial E_x}{\partial t} &= \frac{1}{\epsilon_o}\frac{\partial H_z}{\partial y}, \\ \frac{\partial E_y}{\partial t} &= -\frac{1}{\epsilon_o}\frac{\partial H_z}{\partial x}, \\ \frac{\partial H_z}{\partial t} &= -\frac{1}{\mu_o}\left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} + S\right), \end{aligned} \tag{28}$$

where E_x and E_y denote the in-plane electric field components and H_z denotes the out-of-plane magnetic field component. The constants ϵ_o and μ_o are the permeability and permittivity of the space, respectively. The source function S represents a Gaussian pulse emitted by a point source located at (x_o, y_o), which can be expressed as

$$S(x, y, t) = \delta(x - x_o)\delta(y - y_o) \cdot \exp\Big(- \Big(\frac{t - \tau}{\omega}\Big)^2\Big), \tag{29}$$

with temporal delay τ and pulse width ω. The spatial point source is approximated using the Gaussian distribution centered at (x_o, y_o) with kernel width $\sigma = 0.01$.

$$\delta(x - x_0)\,\delta(y - y_0) \approx \frac{1}{2\pi\sigma^2}\exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right). \tag{30}$$

To solve this system of equations, all physical quantities are non-dimensionalized such that ϵ_o and μ_o become unity, and take $\tau = 0.25$ and $\omega = 0.25$. We solve the problem for the domain $\Omega = [0, 1]^2$ with the point source located at $(x_o, y_o) = (0.5, 0.5)$ and starting with a null field condition. The simulation is run for the non-dimensional time $T = 0.5$. For a reference solution, we employ the finite-difference time-domain (FDTD) method with spatial resolution as $\Delta x = \Delta y = 5 \times 10^{-3}$ and time step $\Delta t = 1.5 \times 10^{-3}$, chosen to satisfy the CFL stability condition under the non-dimensional wave speed.

W-PINN and MMPINN are unable to solve this problem satisfactorily, which is evident from high relative L_2-error values in the Table 4.4. Further, the point-wise error plot shown in Fig. 4.7 demonstrates higher accuracy of AW-PINN prediction.

Table 4.4 Comparison of various methods for solving Maxwell's equation 4.4.

Method	Relative L_2-Error E_x	Relative L_2-Error E_y	Relative L_2-Error H_z	Avg. Training Time
W-PINN	$8.52 \pm 0.21 \times 10^{-2}$	$8.94 \pm 0.13 \times 10^{-2}$	$4.52 \pm 0.21 \times 10^{-1}$	42.71 min
MMPINN	$4.33 \pm 0.17 \times 10^{-2}$	$3.91 \pm 0.21 \times 10^{-2}$	$2.31 \pm 0.18 \times 10^{-1}$	182.16 min
AW-PINN	$\mathbf{5.31 \pm 0.61 \times 10^{-3}}$	$\mathbf{6.54 \pm 0.47 \times 10^{-3}}$	$\mathbf{9.01 \pm 0.83 \times 10^{-3}}$	**76.57 min**

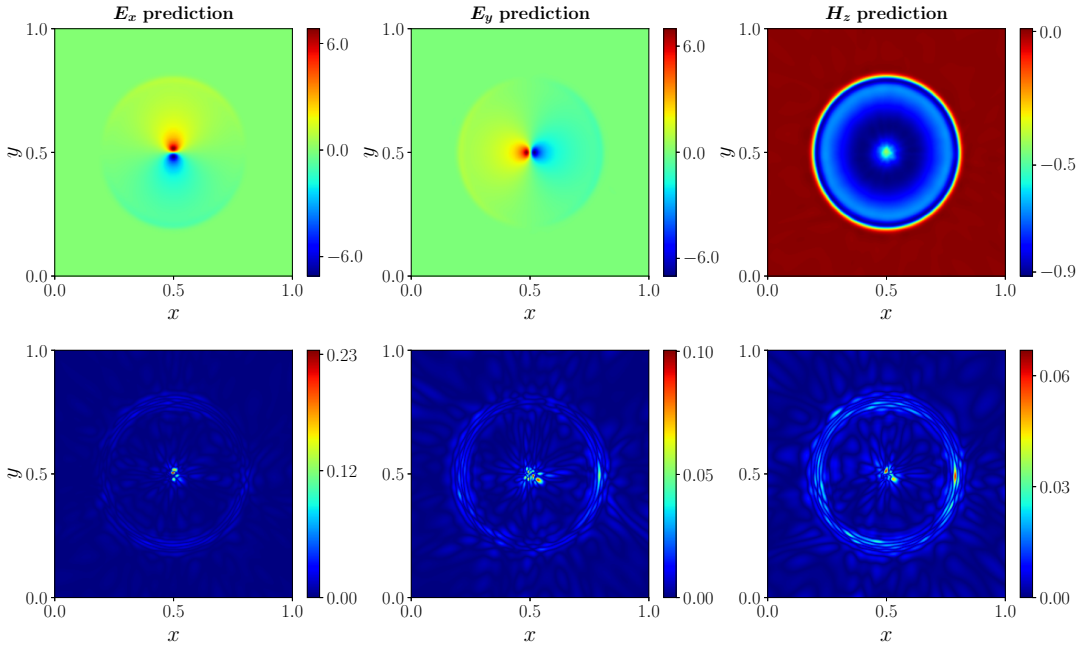

Fig. 4.7. The electromagnetic field prediction using AW-PINN for TEz Maxwell's equations 4.4 at the top panel and their respective point-wise absolute error at the bottom panel.

5 Conclusion

In this study, we addressed the challenges encountered during the training of PINNs due to the severe loss imbalance between different loss components. In particular, we considered problems with localized high-magnitude source terms. For such problems, there is an extreme loss imbalance, and it requires special treatment. To tackle these issues, we proposed an extension of W-PINN that dynamically adapts the wavelet family based on residual and supervised loss functions. This adaptive mechanism enables the model to effectively capture fine-scale features without populating the entire domain with higher-resolution wavelets. Through various challenging problems, we have demonstrated significant improvement over W-PINN and the well-known MMPINN.

Although the method is promising and has achieved impressive accuracy with highly competitive training time, there is still room for improvement. The similarity-based selection of wavelet bases involves manual tuning partially and depends on the quality of the pre-training phase, suggesting that more automated selection schemes could improve robustness. Additionally, systematically investigating the performance of AW-PINN using various wavelet bases could be an interesting comparison. Moreover, future work could involve evaluating the AW-PINN across a broader class of multiscale problems to further assess its general applicability and performance.

Acknowledgment

The authors would like to thank Mr. Anirudha Sen, Department of Mechanical Engineering, Indian Institute of Technology Bhilai, India, for his assistance in computations related to Neural Tangent Kernel theory for Physics-Informed Neural Networks.

Funding details

No funding source is available for this research.

Data availability statements

All source code to reproduce the results of this study will be made available on request.

Conflicts of interest and declarations

The authors declare that they do not have any conflicts of interest. In addition, they also declare that this work is not under consideration anywhere.

References

[1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic differentiation in machine learning: a survey, J. Mach. Learn. Res. 18 (153) (2018) 1–43.
URL http://jmlr.org/papers/v18/17-468.html

[2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, Pytorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, Vol. 32, Curran Associates, Inc., 2019.
URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[3] M. Raissi, P. Perdikaris, G. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.

[4] Z. Hu, K. Shukla, G. E. Karniadakis, K. Kawaguchi, Tackling the curse of dimensionality with physics-informed neural networks, Neural Networks 176 (2024) 106369. doi:10.1016/j.neunet.2024.106369.

[5] F. Sahli Costabal, S. Pezzuto, P. Perdikaris, Δ-pinns: Physics-informed neural networks on complex geometries, Eng. Appl. Artif. Intell. 127 (2024) 107324. doi:10.1016/j.engappai.2023.107324.

[6] L. Yuan, Y.-Q. Ni, X.-Y. Deng, S. Hao, A-pinn: Auxiliary physics informed neural networks for forward and inverse problems of nonlinear integro-differential equations, J. Comput. Phys. 462 (2022) 111260. doi:10.1016/j.jcp.2022.111260.

[7] D. Zhang, L. Guo, G. E. Karniadakis, Learning in modal space: Solving time-dependent stochastic pdes using physics-informed neural networks, SIAM J. Sci. Comput. 42 (2) (2020) A639–A665. doi:10.1137/19M1260141.

[8] Tushar, S. Chakraborty, Deep physics corrector: A physics enhanced deep learning architecture for solving stochastic differential equations, J. Comput. Phys. 479 (2023) 112004. doi:10.1016/j.jcp.2023.112004.

[9] G. Pang, L. Lu, G. E. Karniadakis, fpinns: Fractional physics-informed neural networks, SIAM J. Sci. Comput. 41 (4) (2019) A2603–A2626. doi:10.1137/18M1229845.

[10] S. S. M., P. Kumar, V. Govindaraj, A novel optimization-based physics-informed neural network scheme for solving fractional differential equations, Engineering with Computers 40 (2) (2024) 855–865. doi:10.1007/s00366-023-01830-x.

[11] S. Cuomo, V. S. D. Cola, F. Giampaolo, G. Rozza, M. Raissi, F. Piccialli, Scientific machine learning through physics–informed neural networks: Where we are and what's next, J. Sci. Comput. 92 (3) (2022) 88. doi:10.1007/s10915-022-01939-z.

[12] W. Zhang, W. Suo, J. Song, W. Cao, Physics informed neural networks (pinns) as intelligent computing technique for solving partial differential equations: Limitation and future prospects, arXiv preprint arXiv:2411.18240 (2024). doi:10.48550/arXiv.2411.18240.

[13] C. Zhao, F. Zhang, W. Lou, X. Wang, J. Yang, A comprehensive review of advances in physics-informed neural networks and their applications in complex fluid dynamics, Phys. Fluids. 36 (10) (2024) 101301. doi:10.1063/5.0226562.

[14] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of neural networks, in: K. Chaudhuri, R. Salakhutdinov (Eds.), Proceedings of the 36th International Conference on Machine Learning, Vol. 97 of Proceedings of Machine Learning

Research, PMLR, 2019, pp. 5301–5310.
URL https://proceedings.mlr.press/v97/rahaman19a.html

[15] Z.-Q. J. Xu, Y. Zhang, T. Luo, Overview frequency principle/spectral bias in deep learning, Commun. Appl. Math. Comput. 7 (3) (2025) 827–864. doi:10.1007/s42967-024-00398-7.

[16] A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: Convergence and generalization in neural networks, in: Advances in Neural Information Processing Systems, Vol. 31, Curran Associates, Inc., 2018.
URL https://proceedings.neurips.cc/paper_files/paper/2018/file/5a4be1fa34e62bb8a6ec6b91d2462f5a-Paper.pdf

[17] S. Wang, X. Yu, P. Perdikaris, When and why pinns fail to train: A neural tangent kernel perspective, J. Comput. Phys. 449 (2022) 110768. doi:https://doi.org/10.1016/j.jcp.2021.110768.

[18] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. 43 (5) (2021) A3055–A3081. doi:10.1137/20M1318043.

[19] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), Advances in Neural Information Processing Systems, Vol. 33, Curran Associates, Inc., 2020, pp. 7537–7547.
URL https://proceedings.neurips.cc/paper_files/paper/2020/file/55053683268957697aa39fba6f231c68-Paper.pdf

[20] S. Wang, H. Wang, P. Perdikaris, On the eigenvector bias of fourier feature networks: From regression to solving multi-scale pdes with physics-informed neural networks, Comput. Methods Appl. Mech. Eng 384 (2021) 113938. doi:https://doi.org/10.1016/j.cma.2021.113938.
URL https://www.sciencedirect.com/science/article/pii/S0045782521002759

[21] Y. Liu, H. Gu, X. Yu, P. Qin, Diminishing spectral bias in physics-informed neural networks using spatially-adaptive fourier feature encoding, Neural Networks 182 (2025) 106886. doi:https://doi.org/10.1016/j.neunet.2024.106886.
URL https://www.sciencedirect.com/science/article/pii/S0893608024008153

[22] L. D. McClenny, U. M. Braga-Neto, Self-adaptive physics-informed neural networks, J. Comput. Phys. 474 (2023) 111722. doi:10.1016/j.jcp.2022.111722.

[23] Y. Wang, Y. Yao, J. Guo, Z. Gao, A practical pinn framework for multi-scale problems with multi-magnitude loss terms, J. Comput. Phys. 510 (2024) 113112. doi:10.1016/j.jcp.2024.113112.

[24] R. Bischof, M. A. Kraus, Multi-objective loss balancing for physics-informed deep learning, Comput. Methods Appl. Mech. Engrg. 439 (2025) 117914. doi:10.1016/j.cma.2025.117914.

[25] H. Pandey, A. Singh, R. Behera, An efficient wavelet-based physics-informed neural networks for singularly perturbed problems (2025). arXiv:2409.11847.
URL https://arxiv.org/abs/2409.11847

[26] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization (2017). arXiv:1412.6980.
URL https://arxiv.org/abs/1412.6980

[27] J. Nocedal, Updating quasi-newton matrices with limited storage, Math. Comp. 35 (151) (1980) 773–782.
URL https://courses.grainger.illinois.edu/ece544na/fa2014/nocedal80.pdf

[28] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Y. W. Teh, M. Titterington (Eds.), Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, Vol. 9 of Proceedings of Machine Learning Research, PMLR, Chia Laguna Resort, Sardinia, Italy, 2010, pp. 249–256.
URL https://proceedings.mlr.press/v9/glorot10a.html

Appendix A: Hyperparameters

Table A.1 Hyperparameters used for MMPINN

Hyperparameter	**Heat Conduction 4.1**			**Poisson Problem 4.2**		**Flow Eq. 4.3**	**Maxwell's Eq. 4.4**
	$\epsilon = 0.12$	$\epsilon = 0.11$	$\epsilon = 0.10$	$\epsilon = 0.05$	$\epsilon = 0.02$		
Hidden layers	5	5	6	5	5	5	8
Neurons per layer	400	400	500	200	300	300	400
Residual points N_{res}	90000	100000	200000	30000	50000	50000	150000
Supervised points N_{sup}	9000	12000	21000	6000	8000	4000	20000
Exponent p	3.0	4.0	4.0	3.0	4.0	3.0	2.0
Exponent q	1.0	1.0	1.0	1.0	1.0	1.0	1.0
ADAM iterations	1000	2000	4000	2000	2000	4000	10000

Table A.2 Hyperparameters used for W-PINN

Hyperparameter	**Heat Conduction 4.1**			**Poisson Problem 4.2**		**Flow Eq. 4.3**	**Maxwell's Eq. 4.4**
	$\epsilon = 0.12$	$\epsilon = 0.11$	$\epsilon = 0.10$	$\epsilon = 0.05$	$\epsilon = 0.02$		
Hidden layers	6	6	6	9	9	6	10
Neurons per layer	100	100	100	50	50	100	200
Residual points N_{res}	20000	20000	20000	10000	15000	20000	40000
Supervised points N_{sup}	3000	3000	3000	4000	4000	2000	10000
Residual loss weight ω_r	0.01	0.01	0.01	0.01	0.01	1.0	0.1
Supervised loss weight ω_s	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Resolution J_x	[-6,5]	[-6,5]	[-6,5]	[-6,6]	[-6,6]	[-6,6]	[-5,4]
Resolution J_y	-	-	-	[-6,6]	[-6,6]	-	[-5,4]
Resolution J_t	[-6,6]	[-6,6]	[-6,6]	-	-	[-6,6]	[-5,4]
Translation parameter γ	0.3	0.3	0.3	0.1	0.2	0.3	0.1
ADAM iterations	1000	2000	2000	1000	2000	3000	10000

Table A.3 Hyperparameters used for AW-PINN

Hyperparameter	**Heat Conduction 4.1**			**Poisson Problem 4.2**		**Flow Eq. 4.3**	**Maxwell's Eq. 4.4**
	$\epsilon = 0.12$	$\epsilon = 0.11$	$\epsilon = 0.10$	$\epsilon = 0.05$	$\epsilon = 0.02$		
Residual points N_{res}	20000	20000	20000	10000	10000	20000	40000
Supervised points N_{sup}	3000	3000	3000	4000	4000	2000	10000
Pre-trainingADAM iterations	1000	2000	3000	1000	2000	1000	10000
Residual loss weight ω_r	0.01	0.01	0.01	0.01	0.01	1.0	0.1
Supervised loss weight ω_s	1.0	1.0	1.0	10.0	10.0	1.0	1.0
Threshold IC	> 0.5	> 0.2	> 0.4	-	-	> 0.5	< 0.95
Threshold BC	< 0.002	< 0.02	< 0.003	> 0.2	> 0.2	> 0.95	< 0.95
Threshold PDE	< 0.980	< 0.988	< 0.984	< 0.850	< 0.975	< 0.25	< 0.91
Top κ-%	8	10	8	2	1	2	5